

Cloud Computing Spring 2026

Standard Project Idea Proposal Template and Supervisor Scoring Checklist

Use this document in two stages: **students fill in Part I, then supervisors review Part II** against the same submission.

Who uses it	Students complete Part I before idea approval. Supervisors complete Part II after reviewing the proposal.
Submission rule	Ideas should be approved only when the problem is clear, the scope is realistic, the course mapping is genuine, the evaluation plan is measurable, and the team can demonstrate a working end-to-end deliverable.

Part I — Student Project Idea Proposal Template

Students should submit all fields below in the same order. If a field does not apply, write “Not Applicable” and justify briefly.

Team name / ID	20230257
Team type	AI-only
Course mapping	Cloud + Computer Vision + Evolutionary Algorithms
Proposed project title	Vision Feature Selection and Neural Evolution Platform using Evolutionary Optimization
Team members and IDs	20230257 سلمى محمد عبدالعزيز امين 20230217 روان عصام الدين فهمي 20230475 محمد سعيد احمد 20230521 محمود حسام الدين محمود 20230029 احمد علاء عبدالرحيم علي 20230629 نور حاتم محمد المهدي 20230662 ياسين ياسر زكريا

1. Problem statement and target users

- What problem are you solving?
- Who are the target users or stakeholders?
- Why is this problem worth solving in this semester project?

Team response:

The project addresses the problem of **improving image classification performance efficiently without relying only on manual tuning or expensive deep learning experimentation.**

Target users are **AI researchers, students, and developers** who want to optimize vision models automatically.

This problem is worth solving because model tuning is time-consuming and computationally expensive, and evolutionary optimization provides a structured way to improve performance within limited resources.

2. Project summary in plain language

- Describe the idea in 4–6 clear sentences without buzzwords.
- State what the system will do from the user's point of view.

Team response:

This system is an image classification platform that first builds a standard deep learning model to recognize images. It then uses evolutionary algorithms to automatically improve the model by selecting better features or tuning parameters.

Users can upload or select a dataset and run experiments to compare baseline and optimized results.

The system shows performance improvements through a dashboard. It also supports storing and visualizing experiment results.

The goal is to make model optimization easier, automated, and visually understandable.

3. Core features or modules

- List the planned modules or subsystems.
- For software-heavy teams, show at least four meaningful functional modules.
- Mark which modules are essential for the minimum viable version.

Team response:

- ☐ Dataset loading and preprocessing module (Essential MVP)
- ☐ Baseline CNN image classification model (Essential MVP)
- ☐ Evolutionary optimization module (GA/DE) (Essential MVP)
- ☐ Experiment evaluation and comparison module (Essential MVP)
- ☐ Cloud storage and deployment module
- ☐ Results visualization dashboard (Optional improvement)

4. Course mapping and academic contribution

- State which course(s) the project maps to.
- Explain what each course contributes to the project.
- For mixed teams, clearly separate the CS layer, AI/algorithm layer, and cloud layer.

Team response:

- **Computer Vision:** Image preprocessing, CNN model training, feature extraction, and evaluation
- **Evolutionary Algorithms:** Genetic Algorithm for feature selection and hyperparameter optimization
- **Cloud Computing:** Deployment, storage of experiments, logging, and scalable execution environment

5. Technical approach and stack

- List the main technologies, frameworks, languages, libraries, and databases.
- State whether the solution is monolithic, modular, or microservices-based.
- Identify which parts will be implemented in Spring Boot, Python, or other tools.

Team response:

- ☐ **Frontend:** Streamlit dashboard
- ☐ **Backend:** Python (FastAPI optional for APIs)
- ☐ **AI/ML:** PyTorch, OpenCV, scikit-learn
- ☐ **Evolutionary Algorithms:** DEAP library (GA/DE implementation)
- ☐ **Database:** MongoDB or PostgreSQL for experiment logs
- ☐ **Architecture:** Modular monolithic system
- ☐ **Cloud/Deployment:** Docker + AWS

6. Data, inputs, or test scenarios

- Identify the dataset, input source, simulated scenarios, or sample cases you will use.
- State how you will obtain or prepare them.
- Note any data limitations, privacy concerns, or access restrictions.

Team response:

The system will use public datasets such as:

- CIFAR-10 (60,000 images, 10 classes)
- MNIST (70,000 handwritten digit images)

These datasets are publicly available and require no privacy restrictions.

Inputs include image data for classification and experimental configuration settings for optimization runs.

7. AI / model / algorithm plan (complete if applicable)

- State the baseline model or baseline algorithm.
- State the intended improvement, optimization, or experimental comparison.
- For NLU-like ideas: identify the base model, LoRA plan, and structured output format.
- For EA-like ideas: define representation, fitness function, operators, and experiment settings.

Team response:

Baseline model: Simple CNN trained on CIFAR-10 or MNIST

Improvement method: Genetic Algorithm applied to optimize:

- Feature selection OR
- CNN hyperparameters (learning rate, filters, layers)

Fitness function: Classification accuracy on validation set

Experiments will compare baseline vs optimized model performance

--

8. Cloud and deployment plan

- State how the project will use cloud resources.
- Identify deployment targets, storage needs, monitoring/logging needs, and security considerations.
- Explain why the cloud contribution is meaningful, not decorative.

Team response:

The system will use cloud computing primarily for **model inference deployment**, not training.

Cloud responsibilities:

- Deploy CNN + GA optimized model using Docker
- Provide remote inference endpoint (image → prediction)
- Store experiment results (optional lightweight database)
- Host Streamlit dashboard for visualization

Cloud platform: AWS

Training will be performed locally to reduce cost and complexity

Cloud is used for deployment, accessibility, and inference serving

9. Evaluation plan

- List the success criteria and metrics.
- Include both functional metrics and technical/model metrics where relevant.
- State what you will compare against: baseline, ablation, heuristic, or previous version.

Team response:

Success will be measured using:

- Accuracy and F1-score improvement
- Comparison between baseline and optimized model
- Training efficiency and runtime
- Stability of experiment execution
- Dashboard correctness and usability

Baseline CNN results will be compared against GA-optimized results.

10. Scope control and minimum viable version

- State the minimum viable version that must work by the final demo.
- List optional stretch features separately.
- State what you will cut first if time becomes tight.

Team response:

Minimum viable version:

- Prepare and preprocess a publicly available image dataset.
- Train a baseline CNN model for image analysis.
- Implement one evolutionary optimization method (**Genetic Algorithm – GA**) for feature selection.
- Deploy the trained model on a cloud platform.
- Enable basic cloud-based access to the model (e.g., running inference through the cloud).

- Compare results between the baseline model and the optimized model.
- Display results using a simple dashboard.

Optional Stretch Features:

- ☐ Add second evolutionary algorithm (PSO or DE)
 - ☐ Store experiment logs in cloud database (MongoDB/PostgreSQL)
 - ☐ Add advanced dashboard analytics
 - ☐ Auto-retraining pipeline in cloud
-

Scope Reduction Plan (If time becomes limited):

If time is limited:

1. Remove advanced dashboard features
2. Remove second evolutionary algorithm
3. Keep only single cloud deployment + inference API

11. Team role split

- List each team member's main responsibility.
- Show how the workload is divided fairly.
- Identify integration points between members or subteams.

Team response:

- Member 1: CNN baseline model development
- Member 2: Genetic Algorithm implementation
- Member 3: Evolutionary algorithm experimentation (DE/variants)
- Member 4: Evaluation and metrics analysis
- Member 5: Data preprocessing and pipeline management
- Member 6: Dashboard development (Streamlit)
- Member 7: Integration + deployment + system coordination

Integration points are handled through Member 7.

12. Demo scenario

- Describe the exact end-to-end demo flow in 5–10 steps.
- State what input the examiner will provide, what the system will do, and what output will be shown.

Team response:

- ☐ User selects dataset (CIFAR-10 or MNIST)
- ☐ System trains baseline CNN model
- ☐ User triggers evolutionary optimization
- ☐ GA selects improved parameters/features
- ☐ Optimized model is trained
- ☐ System compares results
- ☐ Dashboard displays accuracy comparison
- ☐ Graphs show improvement over baseline
- ☐ Experiment logs are stored
- ☐ Final report is generated visually

13. Risks, ethics, and constraints

- List the top risks: data risk, integration risk, compute cost, timeline risk, or dependency risk.
- State how you will reduce each risk.
- Note any ethical, legal, privacy, or safety concerns.

Team response:

Risks:

- High computation cost of evolutionary optimization
- Training time delays
- Integration complexity between modules

- Dataset limitations or slow convergence

Mitigation:

- Use small datasets (CIFAR-10/MNIST)
- Limit optimization scope
- Use early stopping and pre-trained models if needed
- Modular development before integration
- Run experiments incrementally

Ethical considerations:

No privacy risks since only public datasets are used.

Student submission package checklist

- [✓] Proposed project title and clear problem statement
- [✓] Course mapping explained
- [✓] Dataset / input source identified
- [✓] Evaluation plan with metrics
- [✓] MVP scope clearly stated
- [✓] Team role split included
- [✓] Demo scenario included

Part II — Supervisor Scoring Checklist

Use the gate checklist first. Then complete the scoring rubric and final decision block.

A. Approval gate checklist

Approval condition	Yes	No	Notes / conditions
Problem is clearly defined and user value is understandable.	☐	☐	
Scope is realistic for the timeline and team size.	☐	☐	
Project has enough technical depth for the team type.	☐	☐	
Inputs / dataset / scenarios are available and usable.	☐	☐	
Evaluation metrics and comparison plan are clear.	☐	☐	
Work can be divided fairly across team members.	☐	☐	
The project supports a stable end-to-end demo.	☐	☐	
Course mapping is natural and academically defensible.	☐	☐	
Documentation requirements can be satisfied.	☐	☐	
The proposal avoids obvious ethical, legal, or privacy problems.	☐	☐	
Cloud contribution is meaningful and not merely decorative.	☐	☐	
The team has a clear minimum viable version and fallback plan.	☐	☐	

B. Suggested scoring rubric (100 marks)

Criterion	Weight	Score	Supervisor notes
Problem definition and scope	10		
Technical depth and course alignment	15		
System or model design quality	15		
Data / inputs / preprocessing quality	10		
Evaluation plan and measurable success criteria	10		
Feasibility and timeline realism	10		
Team role distribution and fairness	10		
Cloud architecture / deployment relevance	10		
Documentation readiness	5		
Demo readiness	5		

C. Team-type checks

CS-only teams	Must include a real software system with backend logic, APIs, persistence, modular architecture, Dockerization, and meaningful workflows. Reject ideas that are only a basic CRUD app or only a wrapper around external AI APIs.
AI-only teams	Must include a defined modeling or optimization problem, a baseline, measurable evaluation, experiments, and analysis. Reject ideas that are only prompt engineering, only a UI around a ready-made model, or only a literature survey.
Mixed teams	Must show three visible layers: product/system layer, AI or algorithm layer, and cloud/deployment layer. Reject ideas where one layer is only cosmetic or where the work split is clearly unbalanced.

Supervisor note	If the core idea is promising but too broad, use “Approve with conditions” and require a narrower MVP, clearer data plan, or simpler integration scope.
-----------------	---

D. Final decision

Decision	☐ Approve ☐ Approve with conditions ☐ Revise and resubmit ☐ Reject
Required conditions	
Most important scope correction	
Supervisor comments	
Supervisor name / signature / date	